well spot things that you don't. If you don't have colleagues you can ask, ask a friend, or even post the code on the internet (taking care not to upset your company lawyers). If you can't get anybody to review your code for you, or they don't find anything, don't worry—there's still more you can do. For starters, it might be worth leaving the code alone for a while—work on another part of the application, read a book, or go for a walk. If you take a break, your subconscious can work on the problem in the background while you're consciously focused on something else. Also, the code will be less familiar when you come back to it—you might manage to look at it from a different perspective yourself.

An alternative to getting someone else to review your code is to do it yourself. One useful technique is to try to explain how it works *in detail* to someone else. They don't even have to be physically there—many teams have a bear or rubber chicken for this purpose, and I personally find that writing detailed notes can be hugely beneficial. As you explain, think about each line, what could happen, which data it accesses, and so forth. Ask yourself questions about the code, and explain the answers. I find this to be an incredibly powerful technique—by asking myself these questions and thinking carefully about the answers, the problem often reveals itself. These questions can be helpful for *any* code review, not just when reviewing your own code.

Questions to Think about when Reviewing Multithreaded Code

As I've already mentioned, it can be useful for a reviewer (whether the code's author or someone else) to think about specific questions relating to the code being reviewed. These questions can focus the reviewer's mind on the relevant details of the code and can help identify potential problems. The questions I like to ask include the following, though this is most definitely not a comprehensive list. You might find other questions that help you to focus better. Anyway, here are the questions:

- Which data needs to be protected from concurrent access?
- How do you ensure that the data is protected?
- Where in the code could other threads be at this time?
- Which mutexes does this thread hold?